Ben Clemente

2/26/26

Introduction to Programming with Python

Assignment 6

GitHub URL

Functions, Classes, and the Separation of Concerns

Introduction

This document outlines the steps I took to complete Assignment 6. This module focused on improving script organization and professional coding practices by utilizing functions, classes, and the Separation of Concerns (SoC). By moving beyond linear scripts, I learned how to create more modular, maintainable, and scalable code

Organizing Code with Classes and Functions

To improve the script's structure, I utilized the SoC by organizing the logic into two primary classes: FileProcessor and IO. The FileProcessor class was designed to handle all data storage concerns, such as reading from and writing to the Enrollments.json file. The IO class was created to manage the presentation layer, including menu displays and user input.

```
# Processing ------------------------------------- #
class FileProcessor:....
# Presentation ----------------------------------- #
class IO:...
```

Figure 1: Figure 1: Implementation of classes and static methods

Enhanced Error Handling

I integrated structured error handling into a centralized function. I used try-except blocks to catch specific issues, such as a FileNotFoundError when the JSON file is missing or a ValueError when a user enters non-alphabetic characters for a student's name. I created a centralized output_error_messages() function within the IO class. This function provides user-friendly feedback while optionally displaying technical error details for debugging purposes.

```python
@staticmethod  7 usages
def output_error_messages(message: str, error: Exception = None):
    """ This function displays a custom error messages to the user

    ChangeLog: (Who, When, What)
    BClemente,2.24.2026,Created function

    :return: None
    """
    print(message, end="\n\n")
    if error is not None:
        print("-- Technical Error Message -- ")
        print(error, error.__doc__, type(error), sep='\n')
```

Figure 2: Error handling function

Implementing a Menu-Driven Logic in the Main Body

The main body is the section of code located after the class and function definitions. Its primary role is to control the flow of the application by calling the appropriate functions based on user interaction. When the program starts, it immediately calls FileProcessor.read_data_from_file() to load existing student records into memory.

Instead of writing long blocks of repetitive code, each menu option simply triggers a single line of code that calls a complex function.

```python
# Extract the data from the file
students = FileProcessor.read_data_from_file(file_name=FILE_NAME, student_data=students)

# Present and Process the data
while True:

    # Present the menu of choices
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        students = IO.input_student_data(student_data=students)
        continue

    # Present the current data
    elif menu_choice == "2":
        IO.output_student_courses(student_data=students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        IO.output_student_courses(student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop

print("Program Ended")
```

Figure 3: Using my functions in the main body

Summary

Through this assignment, I gained a deeper understanding of how encapsulation and modularity improve code quality. I learned the difference between parameters (the placeholders in a function definition) and arguments (the actual data passed during a call). Most importantly, I discovered how the Separation of Concerns pattern allows for building complex applications that are easy for teams to collaborate on and maintain.